

DQL Queries for Kubernetes

Series: K8S | **Notebook:** 8 of 13 | **Created:** January 2026 | **Last Updated:** 04/04/2026

Advanced Query Patterns for Kubernetes Data

This notebook provides a comprehensive reference of DQL queries for Kubernetes monitoring. From basic entity queries to complex performance analysis, these patterns help you extract insights from your Kubernetes data.

Table of Contents

- 1. [Entity Queries](#)
 - 2. [Metric Queries](#)
 - 3. [Log and Event Queries](#)
 - 4. [Trace Queries](#)
 - 5. [Correlation Queries](#)
 - 6. [Dashboard Queries](#)
 - 7. [Alerting Queries](#)
 - 8. [Query Templates](#)
-

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Kubernetes monitoring

Requirement	Details
Permissions	<code>metrics.read</code> , <code>entities.read</code> , <code>logs.read</code>
Knowledge	K8S-01 through K8S-07
Data	Active Kubernetes cluster monitored

1. Entity Queries

Kubernetes Entity Types

Entity Type	DQL Table	Description
Cluster	<code>dt.entity.kubernetes_cluster</code>	K8s clusters
Node	<code>dt.entity.kubernetes_node</code>	Worker nodes
Namespace	<code>dt.entity.cloud_application_namespace</code>	Namespaces
Workload	<code>dt.entity.cloud_application</code>	Deployments, StatefulSets
Container	<code>dt.entity.container_group_instance</code>	Container instances
Service	<code>dt.entity.service</code>	Detected services

```
// List all Kubernetes clusters
fetch dt.entity.kubernetes_cluster
| fields entity.name, tags
| sort entity.name asc

// Alternative: Smartscape on Grail (entity.name → name)
// smartscapeNodes K8S_CLUSTER
// | fields name, tags
// | sort name asc
```

```
// List all Kubernetes nodes
fetch dt.entity.kubernetes_node
| fields entity.name, tags
| sort entity.name asc

// Alternative: Smartscape on Grail (entity.name → name)
// smartscapeNodes K8S_NODE
// | fields name, tags
// | sort name asc
```

```
// List all namespaces
fetch dt.entity.cloud_application_namespace
| fields entity.name, tags
| sort entity.name asc

// Alternative: Smartscape on Grail (entity.name → name)
// smartscapeNodes K8S_NAMESPACE
// | fields name, tags
// | sort name asc
```

```
// List all workloads (deployments, statefulsets)
fetch dt.entity.cloud_application
| fields entity.name, tags
| sort entity.name asc
| limit 50

// Alternative: Smartscape on Grail (entity.name → name)
// smartscapeNodes K8S_DEPLOYMENT
// | fields name, tags
// | sort name asc
// | limit 50
```

```
// Count Kubernetes nodes
fetch dt.entity.kubernetes_node
| summarize nodeCount = count()
```

2. Metric Queries

Common Kubernetes Metrics

Metric	Description	Unit
<code>dt.kubernetes.container.cpu_usage</code>	Container CPU usage	Millicores
<code>dt.kubernetes.container.memory_working_set</code>	Container memory (active)	Bytes
<code>dt.kubernetes.container.requests_cpu</code>	CPU requests	Millicores
<code>dt.kubernetes.container.requests_memory</code>	Memory requests	Bytes
<code>dt.kubernetes.container.limits_cpu</code>	CPU limits	Millicores
<code>dt.kubernetes.container.limits_memory</code>	Memory limits	Bytes
<code>dt.containers.cpu.throttled_time</code>	CPU throttle time	Nanoseconds
<code>dt.kubernetes.node.cpu_usage</code>	Node CPU usage	Millicores
<code>dt.kubernetes.node.memory_usage</code>	Node memory usage	Bytes

```
// Container CPU usage – top consumers
timeseries avgCpuMillicores = avg(dt.kubernetes.container.cpu_usage),
from:-1h, by:{dt.entity.container_group_instance}
| sort avgCpuMillicores desc
| limit 15
```

```
// Container memory usage approaching limits
timeseries avgMemBytes = avg(dt.kubernetes.container.memory_working_set),
from:-1h, by:{dt.entity.container_group_instance}
| fieldsAdd avgMemBytesValue = arrayAvg(avgMemBytes)
| sort avgMemBytesValue desc
```

```
// CPU throttling detection
timeseries totalThrottle = sum(dt.containers.cpu.throttled_time), from:-1h,
by:{dt.entity.container_group_instance}
| fieldsAdd totalThrottleValue = arrayAvg(totalThrottle)
| filter totalThrottleValue > 0
| sort totalThrottleValue desc
| limit 15
```

```
// Namespace-level resource usage
timeseries avgCpuUsageMillicores = avg(dt.kubernetes.container.cpu_usage),
from:-1h, by:{k8s.namespace.name}
| sort avgCpuUsageMillicores desc
| limit 15
```

```
// Resource requests by namespace (capacity planning)
timeseries avgCpuRequests = avg(dt.kubernetes.workload.requests_cpu),
from:-1h, by:{k8s.namespace.name}
| sort avgCpuRequests desc
| limit 15
```

3. Log and Event Queries

Log Query Patterns

```
// Error logs by namespace
fetch logs, from:-1h
| filter loglevel == "ERROR"
| filter isNotNull(k8s.namespace.name)
| summarize errorCount = count(), by:{k8s.namespace.name}
| sort errorCount desc
| limit 15
```

```
// Kubernetes events - warnings only
fetch logs, from:-1h
| filter matchesPhrase(content, "Warning")
| filter matchesPhrase(log.source, "kubernetes") or matchesPhrase(log.source,
"k8s")
| fields timestamp, content
| sort timestamp desc
| limit 30
```

```
// Pod crashes and restarts
fetch logs, from:-1h
| filter matchesPhrase(content, "CrashLoopBackOff") or matchesPhrase(content, "BackOff")
| fields timestamp, content
| sort timestamp desc
| limit 20
```

```
// OOM events
fetch logs, from:-1h
| filter matchesPhrase(content, "OOMKilled") or matchesPhrase(content, "Out of memory")
| fields timestamp, content
| sort timestamp desc
| limit 20
```

```
// Log volume trend by namespace
fetch logs, from: now() - 24h
| filter isNotNull(k8s.namespace.name)
| summarize log_count = count(), by:{k8s.namespace.name, time_bucket = bin(timestamp, 1h)}
| sort time_bucket asc
| limit 200
```

4. Trace Queries

Span Queries for Kubernetes Services

```
// Service response times in K8s
fetch spans, from:-1h
| filter span.kind == "server"
| filter isNotNull(k8s.namespace.name)
| summarize
  p50 = percentile(duration, 50),
  p95 = percentile(duration, 95),
  p99 = percentile(duration, 99),
  by:{k8s.namespace.name, dt.entity.service}
| sort p99 desc
| limit 15
```

```
// Error rate by service
fetch spans, from:-1h
| filter span.kind == "server"
| filter isNotNull(k8s.namespace.name)
| summarize
    total = count(),
    errors = countIf(otel.status_code == "ERROR"),
    by:{k8s.namespace.name, dt.entity.service}
| fieldsAdd errorRate = 100.0 * toDouble(errors) / toDouble(total)
| filter errors > 0
| sort errorRate desc
| limit 15
```

```
// Slow traces (>1 second)
fetch spans, from:-1h
| filter span.kind == "server" and duration > 1000000000
| filter isNotNull(k8s.namespace.name)
| fields timestamp, trace.id, k8s.namespace.name, span.name, duration
| sort duration desc
| limit 20
```

```
// Request throughput by namespace
fetch spans, from:-1h
| filter span.kind == "server"
| filter isNotNull(k8s.namespace.name)
| summarize requestCount = count(), by:{k8s.namespace.name}
| sort requestCount desc
| limit 10
```

5. Correlation Queries

Joining Multiple Data Sources

```
// High CPU workloads with their names
timeseries avgCpuMillicores = avg(dt.kubernetes.container.cpu_usage),
from:-1h, by:{dt.entity.cloud_application}
| fieldsAdd avgCpuMillicoresValue = arrayAvg(avgCpuMillicores)
| lookup [fetch dt.entity.cloud_application | fields id, entity.name],
sourceField:dt.entity.cloud_application, lookupField:id
| fields entity.name, avgCpuMillicoresValue
| sort avgCpuMillicoresValue desc
```

```
// Nodes with high utilization
timeseries avgNodeCpu = avg(dt.kubernetes.node.cpu_usage), from:-1h, by:
{dt.entity.kubernetes_node}
| fieldsAdd avgNodeCpuValue = arrayAvg(avgNodeCpu)
| filter avgNodeCpuValue > 70
| lookup [fetch dt.entity.kubernetes_node | fields id, entity.name],
sourceField:dt.entity.kubernetes_node, lookupField:id
| fields entity.name, avgNodeCpuValue
| sort avgNodeCpuValue desc
```

6. Dashboard Queries

Queries Optimized for Dashboards

```
// Cluster summary tile
fetch dt.entity.kubernetes_cluster
| summarize clusterCount = count()
```

```
// Node count tile
fetch dt.entity.kubernetes_node
| summarize nodeCount = count()
```

```
// Error rate trend (chart)
fetch logs, from: now() - 24h
| filter loglevel == "ERROR" and isNotNull(k8s.namespace.name)
| summarize errors = count(), by:{time_bucket = bin(timestamp, 1h)}
| sort time_bucket asc
```

```
// Top namespaces by CPU (bar chart)
timeseries avgCpuMillicores = avg(dt.kubernetes.container.cpu_usage),
from:-1h, by:{k8s.namespace.name}
| sort avgCpuMillicores desc
| limit 10
```


7. Alerting Queries

Threshold-Based Alerting Patterns

```
// High memory usage alert query
timeseries avgMemBytes = avg(dt.kubernetes.container.memory_working_set),
from:-1h, by:{dt.entity.container_group_instance}
| fieldsAdd avgMemBytesValue = arrayAvg(avgMemBytes)
```

```
// CrashLoop detection (last hour)
fetch logs, from: now() - 1h
| filter matchesPhrase(content, "CrashLoopBackOff")
| summarize crashCount = count()
```

```
// Error rate spike detection
fetch logs, from: now() - 1h
| filter loglevel == "ERROR" and isNotNull(k8s.namespace.name)
| summarize errors = count(), by:{k8s.namespace.name}
| filter errors > 100
| sort errors desc
```

8. Query Templates

Reusable Query Patterns

Filter by Namespace:

```
| filter k8s.namespace.name == "NAMESPACE_NAME"
```

Filter by Cluster:

```
| filter kubernetesClusterName == "CLUSTER_NAME"
```

Time-based Analysis:

```
fetch logs, from: now() - DURATION
| summarize log_count = count(), by:{time_bucket = bin(timestamp, INTERVAL)}
| sort time_bucket asc
```

Top N Pattern:

```
| sort METRIC desc
| limit N
```

Summary

This notebook provided DQL query patterns for:

- Entity discovery and relationships
- Container and node metrics
- Log and event analysis
- Distributed tracing
- Multi-source correlation
- Dashboard visualizations
- Alerting conditions

References

- [DQL Documentation](#)
- [Kubernetes Metrics Reference](#)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.